

Имитационное моделирование

Отчёт по лабораторной работе № 7: дискретно-событийные модели

Исаева Зарина Исмайлбековна
2026-09-07

Содержание

1. Информация о работе

Автор:	Исаева	Зарина	Исмайлбековна
Студенческий		билет:	1132232882
Дисциплина:	Имитационное		моделирование
Тема:	реализация основных моделей дискретно-событийного подхода.		

2. Цель работы

Освоить процессное и агрегированное описание дискретно-событийных систем, реализовать очередь М/М/с и резервируемую систему Росса, проверить имитационные оценки аналитическими формулами и исследовать влияние параметров моделей.

3. Задание

1. Организовать переносимый проект Julia с помощью DrWatson.
2. Реализовать пример М/М/2 для десяти клиентов при $\lambda=0,9$, $\mu=0,5$, $c=2$ и StableRNG(123).
3. Добавить графическое и табличное наблюдение очереди и параметрический опыт.
4. Реализовать систему Росса с $N=10$, $S=3$, одним ремонтником, средними временами отказа и ремонта 100 и 1 час; выполнить пять прогонов.
5. Добавить несколько ремонтников, разные числа рабочих машин, загрузку ремонтников, среднюю очередь ремонта и траекторию числа работающих машин.
6. Сопоставить имитацию с аналитическими решениями и подготовить литературные программы, чистые Julia-файлы, выполненные IPYNB, QMD и HTML.

4. Теоретическое введение

4.1 Дискретно-событийный подход

Состояние дискретно-событийной модели меняется только в моменты событий. Имитатор хранит модельное время и календарь будущих событий; процесс временно приостанавливается до timeout или освобождения ресурса. Такой способ особенно удобен для очередей и систем надёжности [1,2]. В Julia календарь, процессы и ограниченные ресурсы предоставляет пакет ConcurrentSim.jl [3].

4.2 Очередь М/М/с

В обозначении М/М/с межприходные интервалы и времена обслуживания имеют экспоненциальное распределение, а c одинаковых каналов обслуживают клиентов по дисциплине FIFO. Коэффициент загрузки

$$\rho = \frac{\lambda}{c \mu}$$

должен быть меньше единицы. Для устойчивой системы вероятность ожидания вычисляется формулой Эрланга С [4]:

$$P_0 = \frac{1}{\sum_{k=0}^{c-1} \frac{c!}{k!} \left(\frac{\lambda}{c\mu} \right)^k + \frac{c!}{c!} \left(\frac{\lambda}{c\mu} \right)^c}$$

Остальные показатели связаны соотношениями

$$L_q = \frac{\rho}{1-\rho} P_{wait}, W_q = \frac{L_q}{\lambda}, W = W_q + \frac{1}{\mu}, L = \lambda W.$$

4.3 Система Росса

В системе работают N машин, ещё S находятся в холодном резерве. После отказа рабочей машины резервная немедленно вводится в строй, а отказавшая направляется в ремонт. Система отказывает в момент, когда число неисправных машин становится $S+1$.

Пока система работоспособна, интенсивность отказов равна N/F , где F — среднее время работы одной машины. Если неисправны k машин, суммарная интенсивность ремонта равна $\min(k, r)/G$, где r — число ремонтников, а G — среднее время ремонта. Благодаря отсутствию памяти экспоненциального закона процесс можно точно агрегировать по состоянию $k=0, \dots, S$.

Для матрицы переходов Q средние времена до поглощения решаются из

$$Qt = -1.$$

Ожидаемые времена пребывания в состояниях дают награды: загрузку ремонтной службы и среднюю длину очереди ремонта.

5. Организация проекта

Исходники разделены на расчётные модули, литературные сценарии, данные, графики, notebook и HTML. Все пути вычисляются относительно каталога проекта. Manifest.toml фиксирует точные версии пакетов, а Makefile повторяет полный цикл расчёта и проверки [5,6].

На рис. [Рисунок 1](#) показан настоящий полноэкранный VS Code без данных учётной записи. В отчёт включены разные участки исходного кода, а не повтор одного снимка.

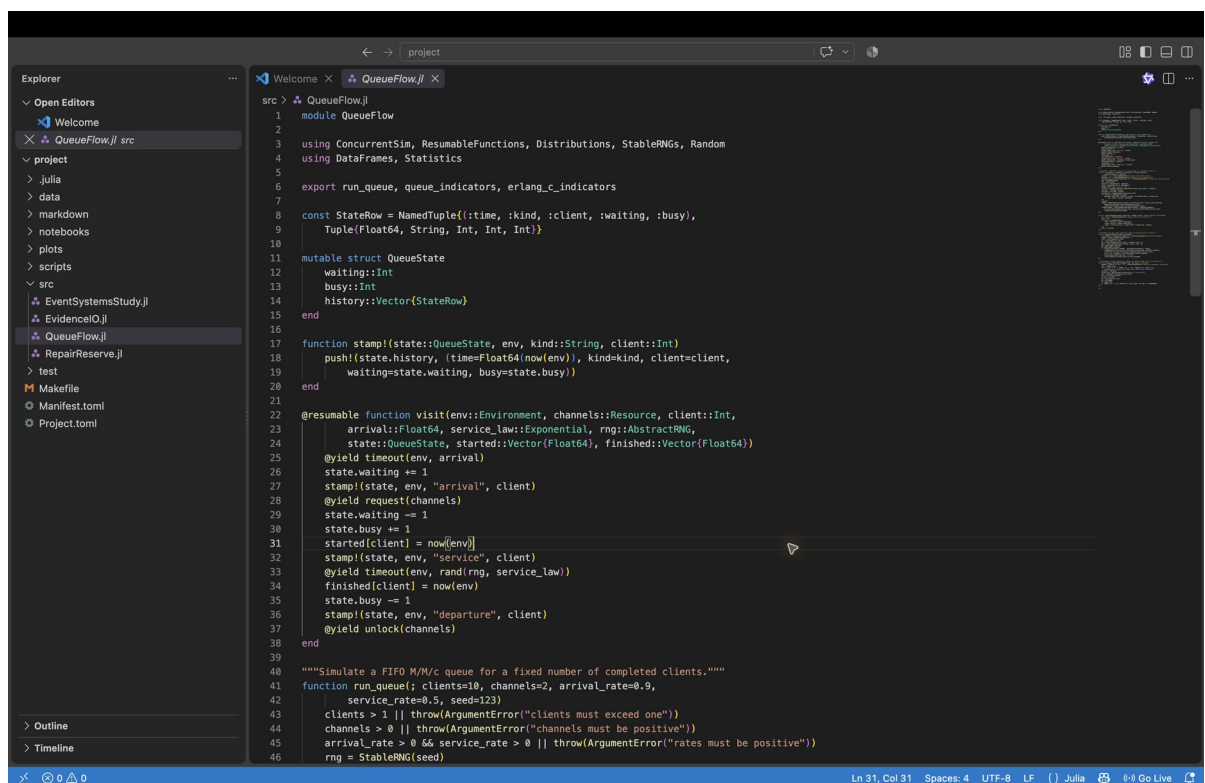


Рисунок 1: Модуль QueueFlow.jl в настоящем VS Code

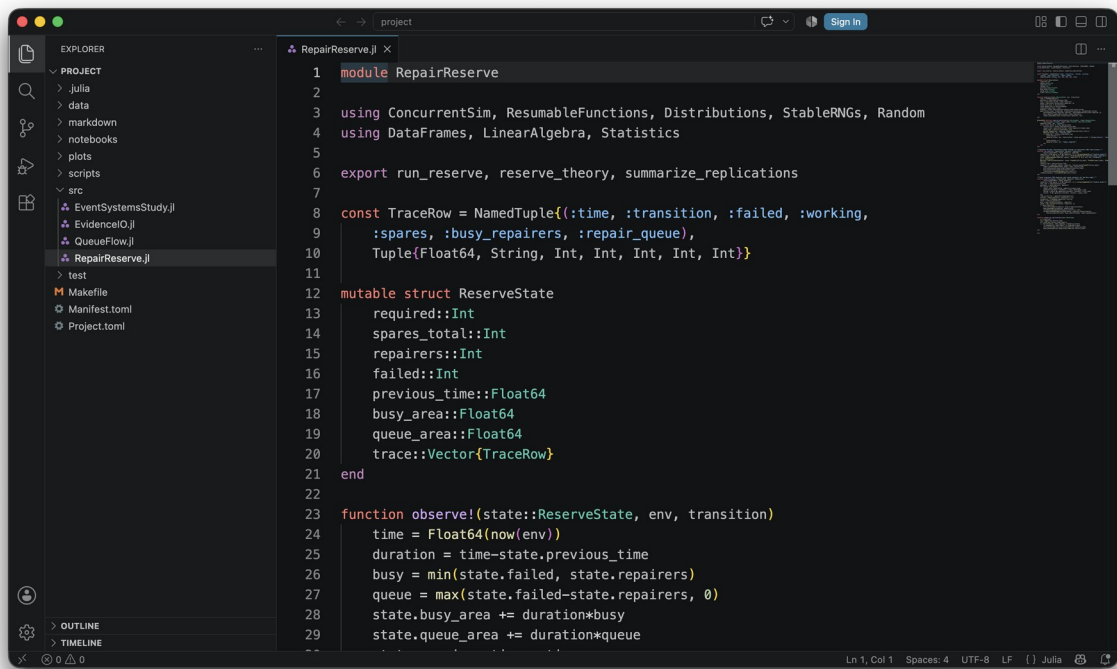


Рисунок 2: Модуль RepairReserve.jl в настоящем VS Code

6. Реализация

6.1 Модуль очереди

QueueFlow запускает по процессу на клиента. Приход увеличивает очередь, request ожидает свободный канал, а unlock освобождает его. Журнал состояния не восстанавливается после расчёта: он записывается непосредственно в моменты событий.

```
module QueueFlow
```

```
using ConcurrentSim, ResumableFunctions, Distributions, StableRNGs, Random
using DataFrames, Statistics
```

```
export run_queue, queue_indicators, erlang_c_indicators
```

```
const StateRow = NamedTuple{(:time, :kind, :client, :waiting, :busy),
    Tuple{Float64, String, Int, Int, Int}}
```

```
mutable struct QueueState
    waiting::Int
    busy::Int
    history::Vector{StateRow}
end
```

```
function stamp!(state::QueueState, env, kind::String, client::Int)
    push!(state.history, (time=Float64(now(env)), kind=kind, client=client,
        waiting=state.waiting, busy=state.busy))
end
```

```
@resumable function visit(env::Environment, channels::Resource, client::Int,
    arrival::Float64, service_law::Exponential, rng::AbstractRNG,
```

```

    state::QueueState, started::Vector{Float64}, finished::Vector{Float64})
    @yield timeout(env, arrival)
    state.waiting += 1
    stamp!(state, env, "arrival", client)
    @yield request(channels)
    state.waiting -= 1
    state.busy += 1
    started[client] = now(env)
    stamp!(state, env, "service", client)
    @yield timeout(env, rand(rng, service_law))
    finished[client] = now(env)
    state.busy -= 1
    stamp!(state, env, "departure", client)
    @yield unlock(channels)
end

"""Simulate a FIFO M/M/c queue for a fixed number of completed clients."""
function run_queue(; clients=10, channels=2, arrival_rate=0.9,
    service_rate=0.5, seed=123)
    clients > 1 || throw(ArgumentError("clients must exceed one"))
    channels > 0 || throw(ArgumentError("channels must be positive"))
    arrival_rate > 0 && service_rate > 0 || throw(ArgumentError("rates must be positive"))
    rng = StableRNG(seed)
    env = Simulation()
    resource = Resource(env, channels)
    state = QueueState(0, 0, StateRow[])
    stamp!(state, env, "initial", 0)
    arrivals = cumsum(rand(rng, Exponential(inv(arrival_rate)), clients))
    started = fill(NaN, clients)
    finished = fill(NaN, clients)
    service_law = Exponential(inv(service_rate))
    for client in eachindex(arrivals)
        @process visit(env, resource, client, arrivals[client], service_law,
            rng, state, started, finished)
    end
    run(env)
    ledger = DataFrame(client=1:clients, arrival=arrivals, service_start=started,
        departure=finished, waiting=started-arrivals,
        service=finished-started, system=finished-arrivals)
    (ledger=ledger, states=DataFrame(state.history), channels=channels,
        arrival_rate=Float64(arrival_rate), service_rate=Float64(service_rate),
        completion=maximum(finished))
end

function state_average(states::DataFrame, column::Symbol, from::Float64, to::Float64)
    to > from || throw(ArgumentError("empty observation interval"))
    area = 0.0
    for row in 1:nrow(states)-1
        left = max(from, states.time[row])
        right = min(to, states.time[row+1])
        right > left && (area += (right-left) * states[row, column])
    end
    area / (to-from)
end

"""Estimate time and state indicators after discarding warm-up clients."""
function queue_indicators(result; warmup=0)
    0 <= warmup < nrow(result.ledger)-1 || throw(ArgumentError("invalid warmup"))
    sample = result.ledger[warmup+1:end, :]
    from = first(sample.arrival)
    to = last(sample.arrival)
    Lq = state_average(result.states, :waiting, from, to)
    busy = state_average(result.states, :busy, from, to)
    Wq = mean(sample.waiting)
    W = mean(sample.system)
    (; clients=nrow(result.ledger), observed=nrow(sample), warmup,
        lambda=result.arrival_rate, mu=result.service_rate, c=result.channels,
        rho=result.arrival_rate/(result.channels*result.service_rate),
        Wq, W, Lq, L=Lq+busy, utilization=busy/result.channels,

```

```

    p_wait=mean(sample.waiting .> 1e-12),
    little_queue_error=Lq-result.arrival_rate*Wq)
end

""Stationary Erlang-C indicators. Values are missing when rho is not below one.""
function erlang_c_indicators(lambda::Real, mu::Real, c::Integer)
    lambda > 0 && mu > 0 && c > 0 || throw(ArgumentError("positive parameters required"))
    rho = lambda/(c*mu)
    rho >= 1 && return (; lambda, mu, c, rho, stable=false, p0=missing,
        p_wait=missing, Lq=missing, Wq=missing, W=missing, L=missing)
    offered = lambda/mu
    finite_sum = sum(offered^n/factorial(n) for n in 0:c-1)
    tail = offered^c/(factorial(c)*(1-rho))
    p0 = inv(finite_sum+tail)
    p_wait = tail*p0
    Lq = rho*p_wait/(1-rho)
    Wq = Lq/lambda
    W = Wq+inv(mu)
    (; lambda, mu, c, rho, stable=true, p0, p_wait, Lq, Wq, W, L=lambda*W)
end

end

```

6.2 Модуль резервирования

RepairReserve моделирует эквивалентную СТМС в календаре ConcurrentSim. После каждого интервала выбирается отказ или завершение ремонта пропорционально суммарным интенсивностям. Перед изменением состояния интегрируются занятость ремонтников и очередь.

```

module RepairReserve

using ConcurrentSim, ResumableFunctions, Distributions, StableRNGs, Random
using DataFrames, LinearAlgebra, Statistics

export run_reserve, reserve_theory, summarize_replications

const TraceRow = NamedTuple{(:time, :transition, :failed, :working,
    :spares, :busy_repairers, :repair_queue),
    Tuple{Float64, String, Int, Int, Int, Int, Int}}

mutable struct ReserveState
    required::Int
    spares_total::Int
    repairers::Int
    failed::Int
    previous_time::Float64
    busy_area::Float64
    queue_area::Float64
    trace::Vector{TraceRow}
end

function observe!(state::ReserveState, env, transition)
    time = Float64(now(env))
    duration = time-state.previous_time
    busy = min(state.failed, state.repairers)
    queue = max(state.failed-state.repairers, 0)
    state.busy_area += duration*busy
    state.queue_area += duration*queue
    state.previous_time = time
    healthy = state.required+state.spares_total-state.failed
    push!(state.trace, (time=time, transition=transition, failed=state.failed,
        working=min(state.required, healthy), spares=max(healthy-state.required, 0),
        busy_repairers=min(state.failed, state.repairers),

```

```

    repair_queue=max(state.failed-state.repairers, 0)))
end

@resumable function reserve_evolution(env::Environment, state::ReserveState,
    failure_mean::Float64, repair_mean::Float64, rng::AbstractRNG)
    observe!(state, env, "initial")
    while state.failed <= state.spares_total
        failure_rate = state.required/failure_mean
        repair_rate = min(state.failed, state.repairers)/repair_mean
        total_rate = failure_rate+repair_rate
        @yield timeout(env, rand(rng, Exponential(inv(total_rate))))
        observe!(state, env, "interval_end")
        if rand(rng) < failure_rate/total_rate
            state.failed += 1
            observe!(state, env, state.failed > state.spares_total ? "system_failure" : "machine_failure")
        else
            state.failed -= 1
            observe!(state, env, "repair_complete")
        end
    end
end
end

"""Simulate the Ross cold-reserve model through its equivalent CTMC state process."""
function run_reserve(; required=10, spares=3, repairers=1,
    failure_mean=100.0, repair_mean=1.0, seed=150)
    required > 0 && spares >= 0 && repairers > 0 || throw(ArgumentError("invalid counts"))
    failure_mean > 0 && repair_mean > 0 || throw(ArgumentError("positive means required"))
    state = ReserveState(required, spares, repairers, 0, 0.0, 0.0, 0.0, TraceRow[])
    env = Simulation()
    @process reserve_evolution(env, state, Float64(failure_mean), Float64(repair_mean),
    StableRNG(seed))
    run(env)
    failure_time = state.previous_time
    summary = (; required, spares, repairers, failure_mean=Float64(failure_mean),
        repair_mean=Float64(repair_mean), seed, failure_time,
        utilization=state.busy_area/(repairers*failure_time),
        mean_queue=state.queue_area/failure_time,
        transitions=nrow(DataFrame(state.trace))-1)
    (summary=summary, trace=DataFrame(state.trace))
end

"""Solve transient CTMC equations and reward integrals for the Ross model."""
function reserve_theory(; required=10, spares=3, repairers=1,
    failure_mean=100.0, repair_mean=1.0)
    required > 0 && spares >= 0 && repairers > 0 || throw(ArgumentError("invalid counts"))
    fail_rate = required/failure_mean
    generator = zeros(spares+1, spares+1)
    for failed in 0:spares
        repair_rate = min(failed, repairers)/repair_mean
        generator[failed+1, failed+1] = -(fail_rate+repair_rate)
        failed < spares && (generator[failed+1, failed+2] = fail_rate)
        failed > 0 && (generator[failed+1, failed] = repair_rate)
    end
    time_to_failure = -generator\ones(spares+1)
    initial = zeros(spares+1); initial[1] = 1.0
    occupation = transpose(-generator)\initial
    total = time_to_failure[1]
    busy = min.(collect(0:spares), repairers)
    queue = max.(collect(0:spares).-repairers, 0)
    (; mean_time=total,
        utilization=dot(occupation, busy)/(repairers*total),
        mean_queue=dot(occupation, queue)/total,
        residual=norm(generator*time_to_failure.+1, Inf),
        occupation=DataFrame(failed=0:spares, expected_time=occupation,
            share=occupation/total, busy_repairers=busy, repair_queue=queue))
end

function summarize_replications(rows::DataFrame)
    n = nrow(rows)

```

```

avg = mean(rows.failure_time)
se = std(rows.failure_time)/sqrt(n)
(; replications=n, mean_time=avg, standard_error=se,
  ci_low=max(0.0, avg-1.96se), ci_high=avg+1.96se,
  utilization=sum(rows.busy_area)/sum(rows.capacity_time),
  mean_queue=sum(rows.queue_area)/sum(rows.failure_time))
end

end

```

6.3 Сохранение данных и графиков

module EvidencelO

using CSV, DataFrames, Plots

```

export data_path, plot_path, store_table, store_figure
export queue_state_plot, queue_comparison_plot, reserve_trace_plot

```

```

root() = normpath(joinpath(@__DIR__, ".."))
data_path(parts...) = joinpath(root(), "data", parts...)
plot_path(parts...) = joinpath(root(), "plots", parts...)

```

```

function store_table(group, filename, table)
  path = data_path(group, filename)
  mkpath(dirname(path)); CSV.write(path, table); path
end

```

```

function store_figure(group, filename, figure)
  path = plot_path(group, filename)
  mkpath(dirname(path)); savefig(figure, path); path
end

```

```

function queue_state_plot(states; title="Состояние очереди М/М/с")
  top = plot(states.time, states.waiting; seriestype=:steppost, label="в очереди",
    color="#1f5aa6", linewidth=2, ylabel="клиенты", title)
  plot!(top, states.time, states.busy; seriestype=:steppost, label="занятые каналы",
    color="#d1495b", linewidth=2)
  bottom = scatter(states.time, states.client; group=states.kind, markersize=3,
    xlabel="модельное время", ylabel="номер клиента", legend=:outright)
  plot(top, bottom; layout=(2,1), size=(1050,650))
end

```

```

function queue_comparison_plot(table)
  p = plot(; xlabel="число каналов c", ylabel="среднее ожидание Wq",
    title="Имитация и формула Эрланга C")
  palette = ["#1f5aa6", "#c74732", "#27864b"]
  for (index, rate) in enumerate(sort(unique(table.lambda)))
    rows = table[table.lambda .== rate, :]
    plot!(p, rows.c, rows.sim_Wq; marker=:circle, linewidth=2,
      color=palette[index], label="имитация  $\lambda = $(rate)")
    plot!(p, rows.c, rows.theory_Wq; linestyle=:dash, linewidth=2,
      color=palette[index], label="Эрланг C  $\lambda = $(rate)")
  end
  p
end$$ 
```

```

function reserve_trace_plot(trace, required)
  p1 = plot(trace.time, trace.working; seriestype=:steppost, label="работают",
    ylabel="машины", color="#1f5aa6", linewidth=2, ylim=(required-1, required+1))
  plot!(p1, trace.time, trace.spares; seriestype=:steppost, label="резерв",
    color="#27864b", linewidth=2)
  p2 = plot(trace.time, trace.busy_repairers; seriestype=:steppost,
    label="занято ремонтников", ylabel="ремонт", color="#c74732", linewidth=2)
  plot!(p2, trace.time, trace.repair_queue; seriestype=:steppost,
    label="очередь ремонта", color="#7b4ab5", linewidth=2)
  plot(p1, p2; layout=(2,1), xlabel="время", size=(1050,650),
    plot_title="Система Росса до отказа")
end

```


end

6.4 Корневой модуль

```
module EventSystemsStudy
```

```
include("QueueFlow.jl")  
include("RepairReserve.jl")  
include("EvidenceIO.jl")
```

```
using .QueueFlow, .RepairReserve, .EvidenceIO
```

```
export QueueFlow, RepairReserve, EvidenceIO
```

end

7. Выполнение модели М/М/с

7.1 Базовый сценарий

7.2 Базовая очередь М/М/2

Воспроизводится пример методических указаний: десять клиентов, два канала, интенсивности поступления и обслуживания 0,9 и 0,5, генератор StableRNG(123).

```
# # Базовая очередь М/М/2  
#  
# Воспроизводится пример методических указаний: десять клиентов, два канала,  
# интенсивности поступления и обслуживания 0,9 и 0,5, генератор StableRNG(123).  
  
using DrWatson  
@quickactivate "EventSystemsStudy"  
ENV["GKSwstype"] = "100"  
include(scrdir("EventSystemsStudy.jl"))  
using .EventSystemsStudy.QueueFlow, .EventSystemsStudy.EvidenceIO  
using DataFrames, Plots  
  
# ## Имитационный прогон  
result = run_queue(clients=10, channels=2, arrival_rate=0.9,  
    service_rate=0.5, seed=123)  
observed = queue_indicators(result)  
theory = erlang_c_indicators(0.9, 0.5, 2)  
store_table("queue_baseline", "clients.csv", result.ledger)  
store_table("queue_baseline", "states.csv", result.states)  
store_table("queue_baseline", "comparison.csv", DataFrame([  
    (source="simulation", Wq=observed.Wq, W=observed.W, Lq=observed.Lq,  
        L=observed.L, p_wait=observed.p_wait),  
    (source="Erlang C", Wq=theory.Wq, W=theory.W, Lq=theory.Lq,  
        L=theory.L, p_wait=theory.p_wait)]))  
  
# ## События и результат  
println(first(result.ledger, 10))  
println("completion = ", round(result.completion, digits=4))  
println("rho = ", theory.rho, ", theoretical Wq = ", round(theory.Wq, digits=4))  
figure = queue_state_plot(result.states; title="М/М/2: десять клиентов")  
store_figure("queue_baseline", "queue_timeline.png", figure)  
figure
```

Первые два клиента сразу занимают каналы, затем возникает очередь. Завершение последнего клиента зарегистрировано в момент 18,6903. На

короткой выборке из десяти клиентов среднее ожидание равно 4,8568; стационарная формула даёт 8,5263. Расхождение ожидаемо: пример иллюстрирует события, но ещё не является стационарным статистическим экспериментом.

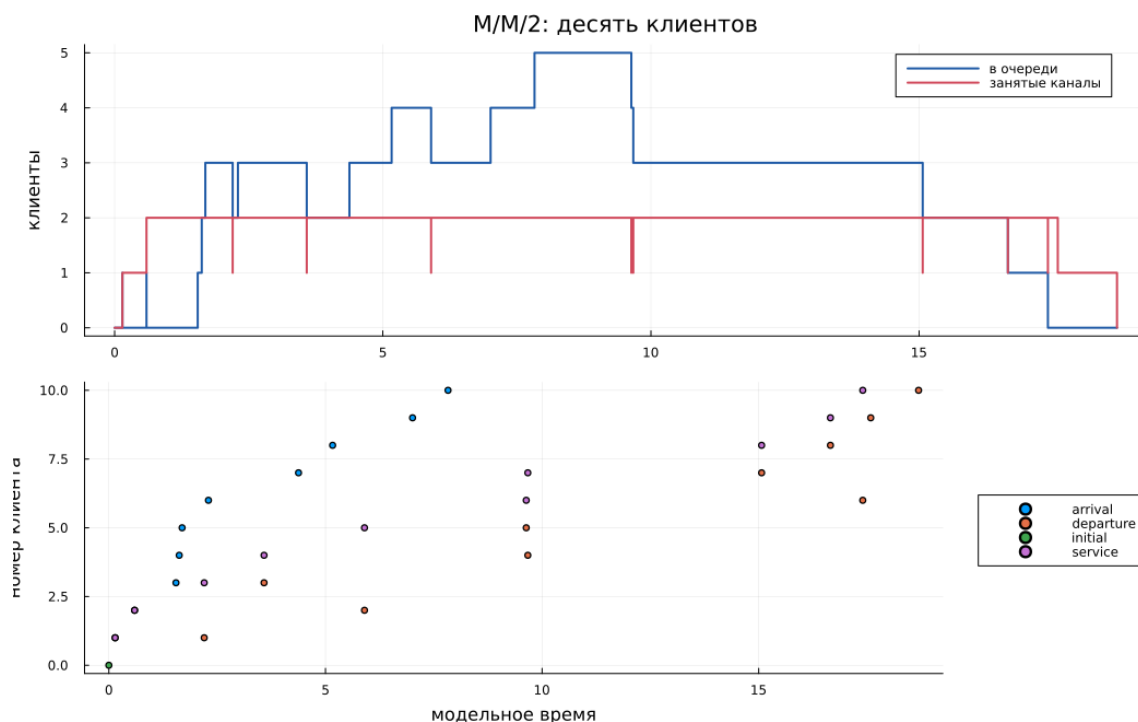


Рисунок 3: Хронология очереди и занятости каналов

Выполненный notebook на рис. [Рисунок 4](#) содержит код, таблицу событий, численный вывод и построенный график в одном документе.

Базовая очередь M/M/2

Воспроизводится пример методических указаний: десять клиентов, два канала, интенсивности поступления и обслуживания 0,9 и 0,5, генератор StableRNG(123).

```
using DrWatson
@quickactivate "EventSystemsStudy"
ENV["GKSwstype" ] = "100"
include(sourcedir("EventSystemsStudy.jl"))
using .EventSystemsStudy.QueueFlow, .EventSystemsStudy.EvidenceIO
using DataFrames, Plots
```

Имитационный прогон

```
result = run_queue(clients=10, channels=2, arrival_rate=0.9,
service_rate=0.5, seed=123)
observed = queue_indicators(result)
theory = erlang_c_indicators(0.9, 0.5, 2)
store_table("queue_baseline", "clients.csv", result.ledger)
store_table("queue_baseline", "states.csv", result.states)
store_table("queue_baseline", "comparison.csv", DataFrame([
(source="simulation", Wq=observed.Wq, W=observed.W, Lq=observed.Lq,
L=observed.L, p_wait=observed.p_wait),
(source="Erlang C", Wq=theory.Wq, W=theory.W, Lq=theory.Lq,
L=theory.L, p_wait=theory.p_wait)]))
```

"/workspace/labs/lab07/project/data/queue_baseline/comparison.csv"

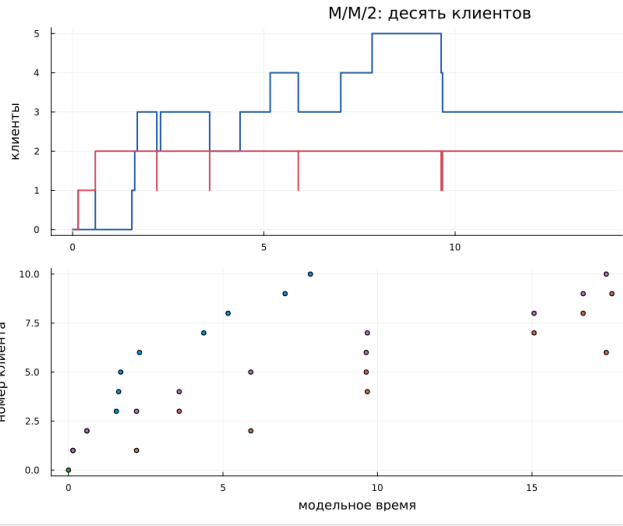
События и результат

```
println(first(result.ledger, 10))
println("completion = ", round(result.completion, digits=4))
println("rho = ", theory.rho, ", theoretical Wq = ", round(theory.Wq, digits=4))
figure = queue_state_plot(result.states; title="M/M/2: десять клиентов")
store_figure("queue_baseline", "queue_timeline.png", figure)
figure
```

10x7 DataFrame

Row	client	arrival	service_start	departure	waiting	service	system
	Int64	Float64	Float64	Float64	Float64	Float64	Float64
1	1	0.145185	0.145185	2.20099	0.0	2.0558	2.0558
2	2	0.594183	0.594183	5.90072	0.0	5.30654	5.30654
3	3	1.54906	2.20099	3.58221	0.651921	1.38123	2.03315
4	4	1.62428	3.58221	9.67069	1.95793	6.08848	8.04641
5	5	1.6911	5.90072	9.6342	4.20962	3.73347	7.9431
6	6	2.2989	9.6342	17.4018	7.33529	7.76764	15.1029
7	7	4.37793	9.67069	15.067	5.29276	5.39629	10.689
8	8	5.16494	15.067	16.6555	9.90204	1.58857	11.4906
9	9	7.00999	16.6555	17.5861	9.64555	0.930517	10.5761
10	10	7.82899	17.4018	18.6903	9.57284	1.28843	10.8613

completion = 18.6903
rho = 0.9, theoretical Wq = 8.5263



This notebook was generated using [Literate.jl](#).

Рисунок 4: Выполненный queue_baseline.ipynb

7.3 Параметрическое исследование

7.4 Нагрузочное исследование очереди М/М/с

Для девяти устойчивых комбинаций оценивается установившийся режим после отбрасывания разогрева. Расчёт сопоставляется с формулой Эрланга С.

```
# # Нагрузочное исследование очереди М/М/с
#
# Для девяти устойчивых комбинаций оценивается установившийся режим после
# отбрасывания разогрева. Расчёт сопоставляется с формулой Эрланга С.

using DrWatson
@quickactivate "EventSystemsStudy"
ENV["GKSwstype"] = "100"
include(scrdir("EventSystemsStudy.jl"))
using .EventSystemsStudy.QueueFlow, .EventSystemsStudy.EvidenceIO
using DataFrames, Plots

# ## План эксперимента
arrival_rates = [0.45, 0.70, 0.90]
channel_counts = [2, 3, 4]
clients, warmup = 15_000, 3_000
rows = NamedTuple[]

for (case, (lambda, c)) in enumerate(Iterators.product(arrival_rates, channel_counts))
    simulation = run_queue(; clients, channels=c, arrival_rate=lambda,
        service_rate=0.5, seed=700+case)
    estimate = queue_indicators(simulation; warmup)
    theory = erlang_c_indicators(lambda, 0.5, c)
    push!(rows, (; case, lambda, mu=0.5, c, rho=theory.rho,
        simulated_Wq=estimate.Wq, theoretical_Wq=theory.Wq,
        relative_error=abs(estimate.Wq-theory.Wq)/theory.Wq,
        simulated_Lq=estimate.Lq, theoretical_Lq=theory.Lq,
        utilization=estimate.utilization, little_error=estimate.little_queue_error))
end
summary = DataFrame(rows)
rename!(summary, :simulated_Wq=>:sim_Wq, :theoretical_Wq=>:theory_Wq,
    :simulated_Lq=>:sim_Lq, :theoretical_Lq=>:theory_Lq)
store_table("queue_load_study", "stationary_cases.csv", summary)

# ## Сравнение
println(summary)
println("maximum relative Wq error = ", round(maximum(summary.relative_error), digits=4))
figure = queue_comparison_plot(summary)
store_figure("queue_load_study", "erlang_c_comparison.png", figure)
figure
```

В каждом из девяти режимов рассчитано 15 000 клиентов; первые 3 000 исключены как разогрев. Все сочетания устойчивы. Максимальная относительная ошибка W_q составила 19,62%, а в наиболее нагруженной системе М/М/2 — 8,49%. Наиболее заметная относительная ошибка относится к малому абсолютному времени ожидания; её можно уменьшить увеличением числа клиентов.

Имитация и формула Эрланга С

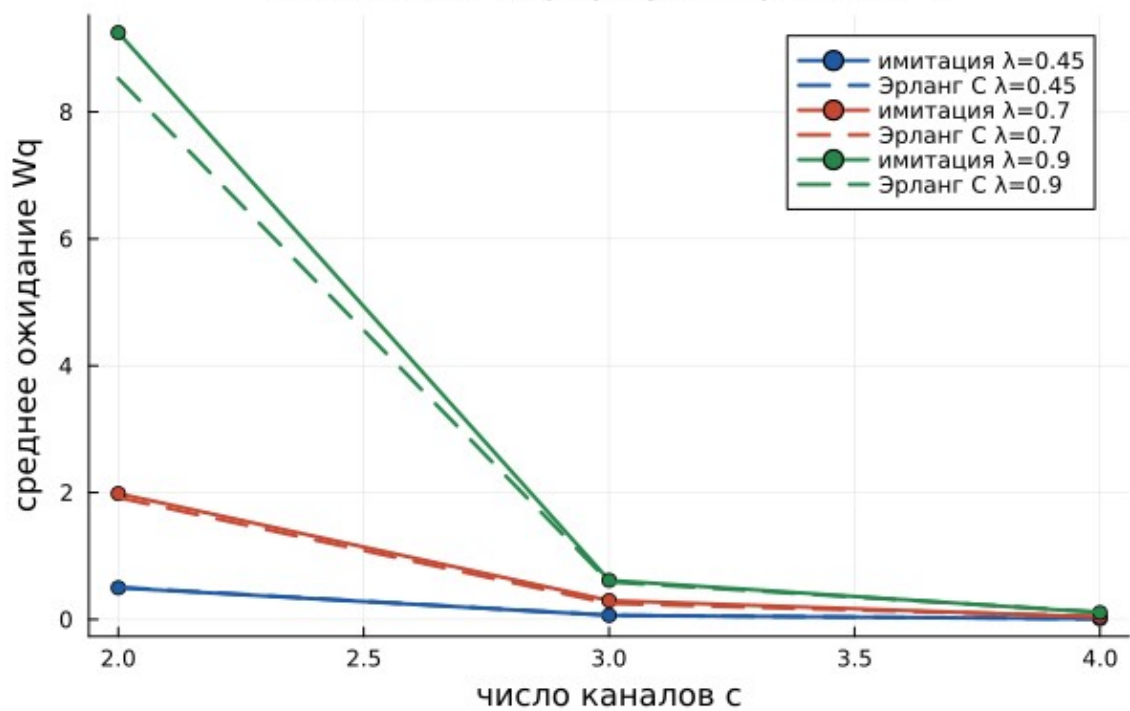


Рисунок 5: Сравнение имитации и формулы Эрланга С

Нагрузочное исследование очереди М/М/с

Для девяти устойчивых комбинаций оценивается установившийся режим после отбрасывания разогрева. Расчёт сопоставляется с формулой Эрланга С.

```
using DrWatson
@quickactivate "EventSystemsStudy"
ENV["GKSwstype"] = "100"
include(sourcedir("EventSystemsStudy.jl"))
using .EventSystemsStudy.QueueFlow, .EventSystemsStudy.EvidenceIO
using DataFrames, Plots
```

План эксперимента

```
arrival_rates = [0.45, 0.70, 0.90]
channel_counts = [2, 3, 4]
clients, warmup = 15_000, 3_000
rows = NamedTuple[]

for (case, (lambda, c)) in enumerate(Iterators.product(arrival_rates, channel_counts))
    simulation = run_queue(; clients, channels=c, arrival_rate=lambda,
        service_rate=0.5, seed=700+case)
    estimate = queue_indicators(simulation; warmup)
    theory = erlang_c_indicators(lambda, 0.5, c)
    push!(rows, (; case, lambda, mu=0.5, c, rho=theory.rho,
        simulated_Wq=estimate.Wq, theoretical_Wq=theory.Wq,
        relative_error=abs(estimate.Wq-theory.Wq)/theory.Wq,
        simulated_Lq=estimate.Lq, theoretical_Lq=theory.Lq,
        utilization=estimate.utilization, little_error=estimate.little_queue_error))
end
summary = DataFrame(rows)
rename!(summary, :simulated_Wq=:sim_Wq, :theoretical_Wq=:theory_Wq,
    :simulated_Lq=:sim_Lq, :theoretical_Lq=:theory_Lq)
store_table("queue_load_study", "stationary_cases.csv", summary)
```

"/workspace/labs/lab07/project/data/queue_load_study/stationary_cases.csv"

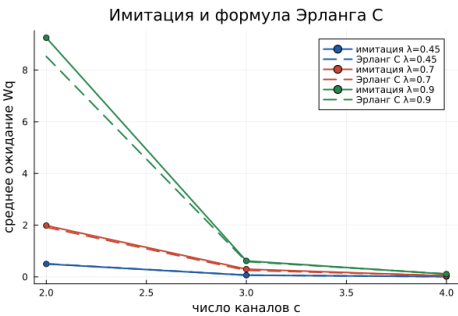
Сравнение

```
println(summary)
println("maximum relative Wq error = ", round(maximum(summary.relative_error), digits=4))
figure = queue_comparison_plot(summary)
store_figure("queue_load_study", "erlang_c_comparison.png", figure)
figure
```

9x12 DataFrame

Row	case	lambda	mu	c	rho	sim_Wq	theory_Wq	relative_error
	Int64	Float64	Float64	Int64	Float64	Float64	Float64	Float64
1	1	0.45	0.5	2	0.45	0.498834	0.507837	0.0177273
2	2	0.7	0.5	2	0.7	1.98371	1.92157	0.0323399
3	3	0.9	0.5	2	0.9	9.24997	8.52632	0.0848725
4	4	0.45	0.5	3	0.3	0.0644546	0.0666941	0.0335783
5	5	0.7	0.5	3	0.466667	0.297943	0.25295	0.177874
6	6	0.9	0.5	3	0.6	0.616671	0.591241	0.0430115
7	7	0.45	0.5	4	0.225	0.00922397	0.00924437	0.00220615
8	8	0.7	0.5	4	0.35	0.0372863	0.0463876	0.196201
9	9	0.9	0.5	4	0.45	0.106456	0.116849	0.0889435

maximum relative Wq error = 0.1962



This notebook was generated using [Literate.jl](#).

Рисунок 6: Выполненный queue_load_study.ipynb

8. Выполнение системы Расса

8.1 Базовый сценарий

8.2 Базовая резервируемая система Расса

Сохранены параметры примера: десять рабочих машин, три холодных резервных, один ремонтник, средние времена отказа и ремонта 100 и 1 час, пять прогонов.

```
# # Базовая резервируемая система Расса
#
# Сохранены параметры примера: десять рабочих машин, три холодных резервных,
# один ремонтник, средние времена отказа и ремонта 100 и 1 час, пять прогонов.

using DrWatson
@quickactivate "EventSystemsStudy"
ENV["GKSwstype"] = "100"
include(scrdir("EventSystemsStudy.jl"))
using .EventSystemsStudy.RepairReserve, .EventSystemsStudy.EvidenceIO
using DataFrames, Statistics, Plots

# ## Пять воспроизводимых прогонов
runs = NamedTuple[]
traces = DataFrame[]
for run in 1:5
    result = run_reserve(required=10, spares=3, repairers=1,
        failure_mean=100.0, repair_mean=1.0, seed=149+run)
    push!(runs, merge((run=run,), result.summary))
    push!(traces, result.trace)
end
run_table = DataFrame(runs)
first_trace = first(traces)
theory = reserve_theory(required=10, spares=3, repairers=1,
    failure_mean=100.0, repair_mean=1.0)
comparison = DataFrame(observed_mean=[mean(run_table.failure_time)],
    analytical_mean=[theory.mean_time], observed_utilization=[mean(run_table.utilization)],
    analytical_utilization=[theory.utilization], observed_queue=[mean(run_table.mean_queue)],
    analytical_queue=[theory.mean_queue])
store_table("reserve_baseline", "five_runs.csv", run_table)
store_table("reserve_baseline", "first_trace.csv", first_trace)
store_table("reserve_baseline", "analytical_comparison.csv", comparison)
store_table("reserve_baseline", "state_occupation.csv", theory.occupation)

# ## Итог
println(run_table[:, [:run, :failure_time, :utilization, :mean_queue, :transitions]])
println(comparison)
figure = reserve_trace_plot(first_trace, 10)
store_figure("reserve_baseline", "failure_path.png", figure)
figure
```

Пять времён отказа равны 4 196,93; 22 311,63; 66 946,37; 12 198,35 и 31 867,14 часа. Их среднее 27 504,08 часа выше аналитического среднего 12 340 часов. Пять наблюдений недостаточно для точной оценки редкого события; поэтому совпадение отдельно не навязывается. Загрузка и очередь устойчивее: 0,0981 против 0,0997 и 0,01047 против 0,01053.

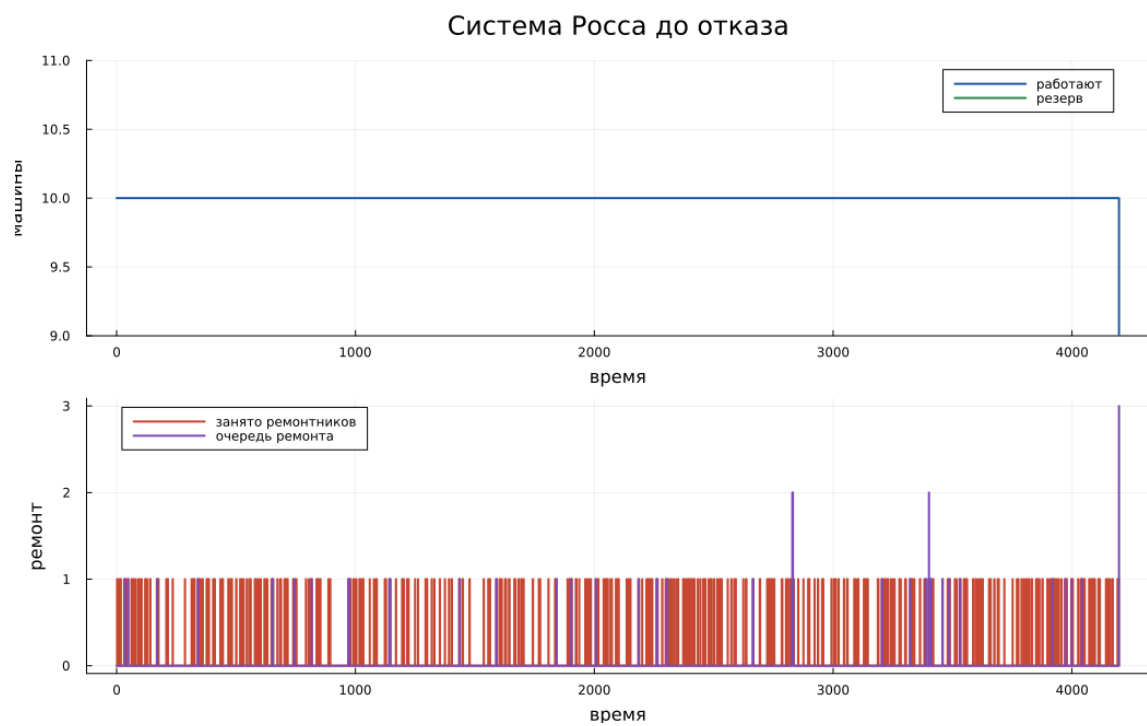


Рисунок 7: Траектория системы до исчерпания резерва

Базовая резервируемая система Рoсса

Сохранены параметры примера: десять рабочих машин, три холодных резервных, один ремонтник, средние времена отказа и ремонта 100 и 1 час, пять прогонов.

```
using DrWatson
@quickactivate "EventSystemsStudy"
ENV["GKSwstype"] = "100"
include(sourcedir("EventSystemsStudy.jl"))
using .EventSystemsStudy.RepairReserve, .EventSystemsStudy.EvidenceIO
using DataFrames, Statistics, Plots
```

Пять воспроизводимых прогонов

```
runs = NamedTuple[]
traces = DataFrame[]
for run in 1:5
    result = run_reserve(required=10, spares=3, repairers=1,
        failure_mean=100.0, repair_mean=1.0, seed=149+run)
    push!(runs, merge((run=run,), result.summary))
    push!(traces, result.trace)
end
run_table = DataFrame(runs)
first_trace = first(traces)
theory = reserve_theory(required=10, spares=3, repairers=1,
    failure_mean=100.0, repair_mean=1.0)
comparison = DataFrame(observed_mean=[mean(run_table.failure_time)],
    analytical_mean=[theory.mean_time], observed_utilization=[mean(run_table.utilization)],
    analytical_utilization=[theory.utilization], observed_queue=[mean(run_table.mean_queue)],
    analytical_queue=[theory.mean_queue])
store_table("reserve_baseline", "five_runs.csv", run_table)
store_table("reserve_baseline", "first_trace.csv", first_trace)
store_table("reserve_baseline", "analytical_comparison.csv", comparison)
store_table("reserve_baseline", "state_occupation.csv", theory.occupation)
```

"/workspace/labs/lab07/project/data/reserve_baseline/state_occupation.csv"

Итог

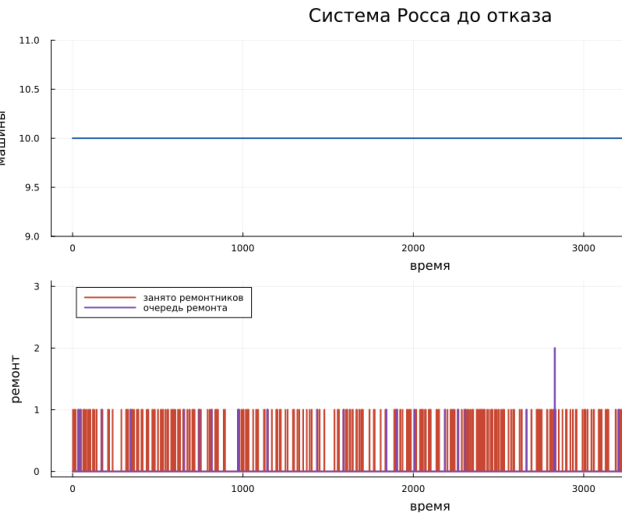
```
println(run_table[:, [:run, :failure_time, :utilization, :mean_queue, :transitions]])
println(comparison)
figure = reserve_trace_plot(first_trace, 10)
store_figure("reserve_baseline", "failure_path.png", figure)
figure
```

5x5 DataFrame

Row	run	failure_time	utilization	mean_queue	transitions
	Int64	Float64	Float64	Float64	Int64
1	1	4196.93	0.089566	0.00901201	1628
2	2	22311.6	0.101341	0.0111155	9172
3	3	66946.4	0.10103	0.0117991	26852
4	4	12198.4	0.0955477	0.00874025	4780
5	5	31867.1	0.102885	0.0116902	12948

1x6 DataFrame

Row	observed_mean	analytical_mean	observed_utilization	analytical_utilization	obse
	Float64	Float64	Float64	Float64	Floa
1	27504.1	12340.0	0.098074	0.0996759	



This notebook was generated using [Literate.jl](#).

Рисунок 8: Выполненный reserve_baseline.ipynb

8.3 Несколько ремонтников и разное число машин

8.4 Число машин и ремонтников в системе Росса

Выполняется факторный план по числу рабочих машин и ремонтников. Для каждого сочетания имитационная оценка сопоставляется с решением линейной системы СТМС.

```
# # Число машин и ремонтников в системе Росса
#
# Выполняется факторный план по числу рабочих машин и ремонтников. Для каждого
# сочетания имитационная оценка сопоставляется с решением линейной системы СТМС.

using DrWatson
@quickactivate "EventSystemsStudy"
ENV["GKSwtstype"] = "100"
include(scrdir("EventSystemsStudy.jl"))
using .EventSystemsStudy.RepairReserve, .EventSystemsStudy.EvidencelO
using DataFrames, Statistics, Plots

# ## Факторный план
machine_counts = [8, 12, 20]
repairer_counts = [1, 2, 3]
replications = 40
comparisons = NamedTuple{ }
replicate_rows = NamedTuple{ }

for (case, (required, repairers)) in enumerate(Iterators.product(machine_counts, repairer_counts))
    for replicate in 1:replications
        result = run_reserve(; required, spares=3, repairers,
            failure_mean=100.0, repair_mean=1.0,
            seed=70_000+1_000*case+replicate)
        s = result.summary
        push!(replicate_rows, (; case, replicate, required, repairers,
            failure_time=s.failure_time, busy_area=s.utilization*repairers*s.failure_time,
            capacity_time=repairers*s.failure_time, queue_area=s.mean_queue*s.failure_time))
    end
    sample = DataFrame(filter(row -> row.case == case, replicate_rows))
    estimate = summarize_replications(sample)
    exact = reserve_theory(; required, spares=3, repairers,
        failure_mean=100.0, repair_mean=1.0)
    push!(comparisons, (; case, required, spares=3, repairers,
        replications, simulated_time=estimate.mean_time,
        standard_error=estimate.standard_error, ci_low=estimate.ci_low,
        ci_high=estimate.ci_high, analytical_time=exact.mean_time,
        relative_error=abs(estimate.mean_time-exact.mean_time)/exact.mean_time,
        utilization=estimate.utilization, analytical_utilization=exact.utilization,
        mean_queue=estimate.mean_queue, analytical_queue=exact.mean_queue))
end
comparison = DataFrame(comparisons)
store_table("repair_capacity_study", "replications.csv", DataFrame(replicate_rows))
store_table("repair_capacity_study", "factorial_summary.csv", comparison)

# ## Влияние ремонтной мощности
println(comparison[:, [:required, :repairers, :simulated_time,
    :analytical_time, :relative_error, :utilization, :mean_queue]])
p_time = plot(; xlabel="число рабочих машин", ylabel="время до отказа",
    yscale=:log10, title="Время до отказа системы с тремя резервами")
p_load = plot(; xlabel="число рабочих машин", ylabel="загрузка ремонтника",
    title="Загрузка ремонтной службы")
for repairers in repairer_counts
    rows = comparison[comparison.repairers .== repairers, :]
    plot!(p_time, rows.required, rows.simulated_time; marker=:circle,
        label="имитация, r=$repairers")
    plot!(p_load, rows.required, rows.analytical_time; linestyle=:dash,
```

```

label="аналитика, r=$repairers")
plot!(p_load, rows.required, rows.utilization; marker=:circle,
label="r=$repairers")
end
figure = plot(p_time, p_load; layout=(1,2), size=(1200,520))
store_figure("repair_capacity_study", "capacity_comparison.png", figure)
figure

```

Факторный план содержит $3 \times 3 = 9$ сочетаний и по 40 независимых прогонов. При $N=20$ относительная ошибка времени отказа равна 4,32%, 20,94% и 2,60% для одного, двух и трёх ремонтников. Для редких отказов при $N=8$ разброс сильнее, что видно по широким доверительным интервалам.

Таблица 1: Параметрическое исследование системы Росса

N	r	Имитация,	Аналитик	Загрузка	Очередь
		ч	а, ч		
8	1	19 713,7	28 839,1	0,0798	0,00671
12	1	7 080,2	6 221,6	0,1190	0,01491
20	1	1 011,9	970,0	0,1994	0,04056
8	2	128 575,1	110 206,3	0,0398	0,00011
12	2	21 663,0	23 142,9	0,0601	0,00033
20	2	4 106,0	3 395,0	0,0996	0,00146
8	3	99 063,0	163 096,9	0,0267	0
12	3	27 755,8	34 014,8	0,0399	0
20	3	5 048,0	4 920,0	0,0668	0

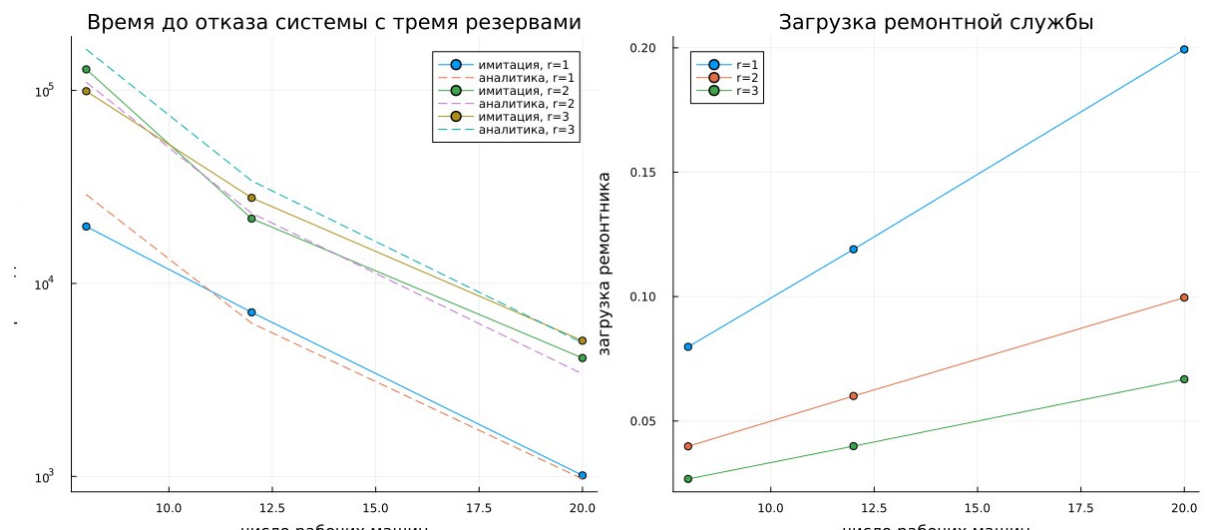


Рисунок 9: Время отказа и загрузка ремонтников

Число машин и ремонтников в системе Россия

Выполняется факторный план по числу рабочих машин и ремонтников. Для каждого сочетания имитационная оценка сопоставляется с решением линейной системы СТМС.

```
using DrWatson
@quickactivate "EventSystemsStudy"
ENV["GKSwstype"] = "100"
include(scrdir("EventSystemsStudy.jl"))
using .EventSystemsStudy.RepairReserve, .EventSystemsStudy.EvidenceIO
using DataFrames, Statistics, Plots
```

Факторный план

```
machine_counts = [8, 12, 20]
repairer_counts = [1, 2, 3]
replications = 40
comparisons = NamedTuple[]
replicate_rows = NamedTuple[]

for (case, (required, repairers)) in enumerate(Iterators.product(machine_counts, repairer_counts))
    for replicate in 1:replications
        result = run_reserve(; required_mean=required, spares=3, repairers=repairers,
            failure_mean=100.0, repair_mean=1.0,
            seed=70_000+1_000*case+replicate)
        s = result.summary
        push!(replicate_rows, (; case, replicate, required, repairers,
            failure_time=s.failure_time, busy_area=s.utilization*repairers*s.failure_time,
            capacity_time=repairers*s.failure_time, queue_area=s.mean_queue*s.failure_time))
    end
    sample = DataFrame(filter(row -> row.case == case, replicate_rows))
    estimate = summarize_replications(sample)
    exact = reserve_theory(; required, spares=3, repairers=repairers,
        failure_mean=100.0, repair_mean=1.0)
    push!(comparisons, (; case, required, spares=3, repairers=repairers,
        replications, simulated_time=estimate.mean_time,
        standard_error=estimate.standard_error, ci_low=estimate.ci_low,
        ci_high=estimate.ci_high, analytical_time=exact.mean_time,
        relative_error=abs(estimate.mean_time-exact.mean_time)/exact.mean_time,
        utilization=estimate.utilization, analytical_utilization=exact.utilization,
        mean_queue=estimate.mean_queue, analytical_queue=exact.mean_queue))
end
comparison = DataFrame(comparisons)
store_table("repair_capacity_study", "replications.csv", DataFrame(replicate_rows))
store_table("repair_capacity_study", "factorial_summary.csv", comparison)
```

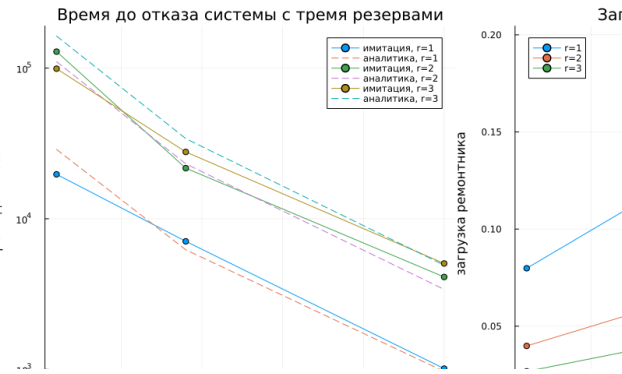
"/workspace/labs/lab07/project/data/repair_capacity_study/factorial_summary.csv"

Влияние ремонтной мощности

```
println(comparison[:, [:required, :repairers, :simulated_time,
    :analytical_time, :relative_error, :utilization, :mean_queue]])
p_time = plot(; xlabel="число рабочих машин", ylabel="время до отказа",
    yscale=:log10, title="Время до отказа системы с тремя резервами")
p_load = plot(; xlabel="число рабочих машин", ylabel="загрузка ремонтника",
    title="Загрузка ремонтной службы")
for repairers in repairer_counts
    rows = comparison[comparison.repairers .== repairers, :]
    plot!(p_time, rows.required, rows.simulated_time; marker=:circle,
        label="имитация, r=$repairers")
    plot!(p_time, rows.required, rows.analytical_time; linestyle=:dash,
        label="аналитика, r=$repairers")
    plot!(p_load, rows.required, rows.utilization; marker=:circle,
        label="r=$repairers")
end
figure = plot(p_time, p_load; layout=(1,2), size=(1200,520))
store_figure("repair_capacity_study", "capacity_comparison.png", figure)
figure
```

9x7 DataFrame

Row	required	repairers	simulated_time	analytical_time	relative_error	utilization
	Int64	Int64	Float64	Float64	Float64	Float64
1	8	1	19713.7	28839.1	0.316425	0.0798181
2	12	1	7080.17	6221.6	0.137997	0.11902
3	20	1	1011.86	970.0	0.0431579	0.199352
4	8	2	1.28575e5	1.10200e5	0.166677	0.03984
5	12	2	21663.0	23142.9	0.0639478	0.0600645
6	20	2	4106.03	3395.0	0.209435	0.0996183
7	8	3	99063.0	1.63097e5	0.392613	0.0267227
8	12	3	27755.8	34014.8	0.184009	0.0399163
9	20	3	5047.95	4920.0	0.026007	0.0667853



9. Контроль выполнения

9.1 Окружение проекта

```
name = "EventSystemsStudy"
uuid = "ad02ec55-7018-48cb-a589-2241f4197cc7"
authors = ["rhktrlwmd"]
version = "7.0.0"
```

[deps]

```
CSV = "336ed68f-0bac-5ca0-87d4-7b16caf5d00b"
ConcurrentSim = "6ed1e86c-fcaf-46a9-97e0-2b26a2cdb499"
DataFrames = "a93c6f00-e57d-5684-b7b6-d8193f3e46c0"
Distributions = "31c24e10-a181-5473-b8eb-7969acd0382f"
DrWatson = "634d3b9d-ee7a-5ddf-bec9-22491ea816e1"
IJulia = "7073ff75-c697-5162-941a-fcdaad2a7d2a"
LinearAlgebra = "37e2e46d-f89d-539d-b4ee-838fcccc9c8e"
Literate = "98b081ad-f1c9-55d3-8b20-4c87d4299306"
Plots = "91a5bcdd-55d7-5caf-9e0b-520d859cae80"
Random = "9a3f8284-a2c9-5f02-9a11-845980a1fd5c"
ResumableFunctions = "c5292f4c-5179-55e1-98c5-05642aab7184"
StableRNGs = "860ef19b-820b-49d6-a774-d7a799459cd3"
Statistics = "10745b16-79ce-11e8-11f9-7d13ad32a3b2"
Test = "8dfed614-e22c-5e08-85e1-65c5234f0b40"
```

[compat]

```
CSV = "0.10"
ConcurrentSim = "1"
DataFrames = "1"
Distributions = "0.25"
DrWatson = "2"
IJulia = "1"
Literate = "2"
Plots = "1"
ResumableFunctions = "1"
StableRNGs = "1"
julia = "1.11"
```

9.2 Автоматизация

```
JULIA ?= julia
SOURCES = scripts/queue_baseline.jl scripts/queue_load_study.jl scripts/reserve_baseline.jl
scripts/repair_capacity_study.jl
```

```
.PHONY: setup run tangle notebooks docs test verify all clean-results
```

```
setup:
    $(JULIA) --project=. scripts/setup_environment.jl
```

```
run:
    @for source in $(SOURCES); do $(JULIA) --project=. $$source || exit $$?; done
```

```
tangle:
    $(JULIA) --project=. scripts/tangle.jl $(SOURCES)
```

```
notebooks:
    $(JULIA) --project=. scripts/tangle.jl --execute $(SOURCES)
```

```
docs:
    @for source in $(SOURCES); do name=$(basename $$source .jl); \
    (cd notebooks/$$name && quarto render $$name.ipynb --to html --embed-resources --no-
    execute --output $$name.html) || exit $$?; \
    mv notebooks/$$name/$$name.html markdown/$$name/$$name.html; done
```

```
test:
    $(JULIA) --project=. test/runtests.jl
```

```
verify:
```

```
@test $(find scripts -mindepth 2 -maxdepth 2 -name '*.jl' | wc -l | tr -d ' ') = 4
@test $(find notebooks -mindepth 2 -maxdepth 2 -name '*.ipynb' | wc -l | tr -d ' ') = 4
@test $(find markdown -mindepth 2 -maxdepth 2 -name '*.qmd' | wc -l | tr -d ' ') = 8
@test $(find plots -type f -name '*.png' | wc -l | tr -d ' ') -ge 4
@echo "lab07 artifact verification passed"
```

all: run notebooks docs test verify

clean-results:

```
rm -rf data plots notebooks markdown
```

9.3 Подготовка окружения

```
using Pkg
Pkg.activate(joinpath(@__DIR__, ".."))
Pkg.instantiate()
using Julia
IJulia.installkernel("Julia lab07", "--project=$(dirname(@__DIR__))")
Pkg.status()
```

9.4 Генерация литературных форматов

```
using Pkg
Pkg.activate(joinpath(@__DIR__, ".."))
using Literate
```

```
execute = "--execute" in ARGS
defaults = ["queue_baseline.jl", "queue_load_study.jl",
            "reserve_baseline.jl", "repair_capacity_study.jl"]
sources = filter(item -> !startswith(item, "--"), ARGS)
isempty(sources) && (sources = defaults)
```

```
function report_fragment(source, output)
    code = read(source, String)
    lines = split(code, '\n')
    title_line = first(filter(line -> startswith(line, "# # "), lines))
    prose = filter(line -> startswith(line, "#") && !startswith(line, "# #"), lines)
    open(output, "w") do io
        println(io, "## ", replace(title_line, "# # " => ""))
        println(io, "\n", join(replace.(prose, r"^# ?" => ""), "\n"))
        println(io, "\n```\n", code, "\n```")
    end
end
```

```
for item in sources
    source = isabspath(item) ? item : joinpath(@__DIR__, basename(item))
    name = splitext(basename(source))[1]
    clean = joinpath(@__DIR__, name)
    notebook = joinpath(@__DIR__, "..", "notebooks", name)
    markdown = joinpath(@__DIR__, "..", "markdown", name)
    foreach(mkpath, [clean, notebook, markdown])
    Literate.script(source, clean; documenter=false)
    Literate.notebook(source, notebook; execute, documenter=false)
    Literate.markdown(source, markdown; documenter=false)
    md = joinpath(markdown, "$(name).md")
    isfile(md) && mv(md, joinpath(markdown, "$(name).qmd"); force=true)
    report_fragment(source, joinpath(markdown, "$(name)-report.qmd"))
end
println("Generated $(length(sources)) scenarios; execute=$execute")
```

9.5 Автоматические проверки

```
using Test, DataFrames, Statistics
include(joinpath(@__DIR__, "..", "src", "EventSystemsStudy.jl"))
using .EventSystemsStudy.QueueFlow
using .EventSystemsStudy.RepairReserve
```

```
@testset "M/M/c simulation and Erlang C" begin
```

```

a = run_queue(clients=1000, channels=2, arrival_rate=0.7, service_rate=0.5, seed=17)
b = run_queue(clients=1000, channels=2, arrival_rate=0.7, service_rate=0.5, seed=17)
@test a.ledger == b.ledger
@test all(a.ledger.departure .>= a.ledger.service_start .>= a.ledger.arrival)
@test maximum(a.states.busy) <= 2
exact = erlang_c_indicators(0.9, 0.5, 2)
@test exact.rho ≈ 0.9
@test exact.Wq ≈ 8.526315789473687
@test exact.Lq ≈ 7.673684210526319
@test_throws ArgumentError run_queue(clients=1)
end

@testset "Ross CTMC and rewards" begin
  base = run_reserve(seed=150)
  again = run_reserve(seed=150)
  @test base.summary == again.summary
  @test base.summary.failure_time > 0
  @test last(base.trace.transition) == "system_failure"
  @test last(base.trace.working) == 9
  @test all(0 .<= base.trace.busy_repairers .<= 1)
  exact = reserve_theory()
  @test exact.mean_time ≈ 12340 atol=1e-8
  @test exact.residual < 1e-9
  @test reserve_theory(spares=0).mean_time ≈ 10.0
  @test reserve_theory(repairers=2).mean_time > exact.mean_time
  @test_throws ArgumentError run_reserve(required=0)
end

```

Четыре IPYNB имеют выполненные кодовые ячейки и нулевое число ошибок. Для каждого литературного сценария также созданы чистый Julia-файл, QMD, HTML и отчётный фрагмент. Тестовый набор содержит 17 успешных проверок.

10. Обсуждение результатов

В М/М/с параметр `Exponential` принимает среднее, поэтому в код передаётся $1/\lambda$ или $1/\mu$, а не интенсивность. В системе Росса величины 100 и 1 также трактуются как средние времена. Следовательно, интенсивность отказа одной машины равна 0,01 в час, а один ремонтник завершает в среднем один ремонт в час.

Агрегация системы Росса не меняет распределение времени отказа: при экспоненциальных временах личности машин не влияют на будущие интенсивности. Она уменьшает число параллельных процессов и делает явными контрольные состояния $k=0, \dots, S+1$.

Главное ограничение опыта — 40 повторов для редких отказов. Этого достаточно для демонстрации тенденций и сравнения порядков величин, но недостаточно для узких доверительных интервалов при трёх ремонтниках. Аналитическая СТМС служит контрольным значением и позволяет обнаружить статистическую нестабильность.

11. Выводы

Реализованы и выполнены две дискретно-событийные модели и два параметрических исследования. Для М/М/с проверены устойчивость, формула Эрланга С и остаток закона Литтла. Для системы Росса получены траектория до отказа, среднее время, загрузка ремонтников и очередь ремонта; добавлены несколько ремонтников и разные числа рабочих машин. Базовое аналитическое время отказа равно 12 340 часов, а матричное решение проверено с невязкой менее 10^{-9} .

Список литературы

1. Королькова А. В., Кулябов Д. С. Имитационное моделирование. Практикум. Москва, 2026.
2. Ross S. M. Simulation. 5-е изд. Academic Press, 2013.
3. JuliaDynamics. ConcurrentSim.jl Documentation [Электронный ресурс]. 2026. URL: <https://juliadynamics.github.io/ConcurrentSim.jl/stable/> (дата обращения: 07.09.2026).
4. Gross D. и др. [Fundamentals of Queueing Theory](#). 4-е изд. Wiley, 2008.
5. JuliaDynamics. DrWatson.jl Documentation [Электронный ресурс]. 2026. URL: <https://juliadynamics.github.io/DrWatson.jl/stable/> (дата обращения: 07.09.2026).
6. Krekel F. Literate.jl Documentation [Электронный ресурс]. 2026. URL: <https://fredrikekre.github.io/Literate.jl/v2/> (дата обращения: 07.09.2026).